

Implementing Interfaces and Programming with Mutable and Immutable Models

Academic week 3

Student: _____

Academic group: _____

Aim and result

This work redesigns a bookstore inventory model so that shared references cannot silently change stock. The solution uses an immutable C# record, ISBN-based equality, price-based ordering, and validated stock operations that return new records. It reproduces the mutable baseline, contrasts default and custom equality, and demonstrates why filtering by value equality removes more than one duplicate entry.

The supplied catalogue is retained in its original order and with its original data. A separate sorted list is used for display and a further copy is used for the duplicate experiment. The application and checks target .NET 10 and use only the standard library.

Required catalogue

Position	Title	ISBN	Price	Stock
1	Refactoring	111	45.00	4
2	Clean Code	222	35.50	2
3	The Pragmatic Programmer	333	40.00	6

Submission contents

The source files, project files, verification script and captured console logs accompany this report. Appendix A contains complete text listings of the application, baseline and checks. The report covers the three tasks, expected and actual results, reproducible commands, and questions for the mandatory oral defence.

Task 1 Immutable records and equality

Why the shared reference prints zero

The statement `Book display = featured;` copies a reference, not the `Book` object. In the starter, `catalog[0]`, `featured` and `display` therefore refer to the same instance. The exact line `display.StockCount = 0;` writes to that instance through a public setter. `Console.WriteLine(featured.StockCount);` reads the same property and prints 0. Mutable means that the object's observable state can be changed after construction.

The starter code is retained in `baseline/OriginalSnippet.txt` with its comment removed. Its pasted ordering places a type declaration before top-level statements, which causes compiler error CS8803 in a standard console project. The runnable `baseline` keeps the same mutable class and executable statements inside `Program.Main`. This packaging adjustment does not change the aliasing behaviour. The actual `baseline` output is 0. [8]

What makes the redesigned model immutable

`Book` is a sealed record class, so it remains a reference type. Merely replacing class with record while retaining public setters would still allow the same bug. Here `Title`, `Isbn` and `Price` have getter-only properties; `StockCount` has a private init accessor. A validated constructor creates the initial state, and only the record's own checked methods can initialize a different stock value in a `with` expression. Strings are immutable and the remaining fields are value types, so the shallow record copy contains no mutable child object. [2, 10]

When `display = featured` is followed by `display = display.Sell(4)`, `display` is rebound to a new record. The original `featured` record and `catalog[0]` keep stock 4; `display` has stock 0. Shared references are harmless because the old object cannot be edited through the public API. The class is sealed to keep the equality contract and construction rules closed to derived record types.

Default equality prediction and measured comparison

Prediction: for `DefaultBook`, the compiler-generated equality compares the runtime record type and every instance field. Its auto-properties store `Title`, `Isbn`, `Price` and `StockCount` in backing fields, so all four values participate. Two default records with ISBN 111 but different price or stock should be unequal. The comparison record in `DefaultBook.cs` checks that prediction by execution. [1]

`Book` implements the record's `IEquatable<Book>` contract by defining `Equals(Book?)` using `StringComparer.Ordinal` on `Isbn` only. `GetHashCode` uses that same comparer and ISBN, which keeps equal records consistent in `HashSet<Book>`. The compiler supplies object equality and equality operators using this custom typed equality. Same ISBN means equal even if title, price and stock differ; another ISBN means unequal. Matching is exact and case-sensitive, with no trimming or ISBN normalization. [1, 3]

Comparison	Expected	Actual
Default records with same ISBN but different price or stock	False	False
Custom records with same ISBN but different other data	True	True
Custom records with different ISBN values	False	False

Complete implementations: Appendix A, `Book.cs` and `DefaultBook.cs`. The supplied `baseline` and executable wrapper are also listed there.

Task 2 Ordering and controlled changes

Ordering without changing the catalogue

Book implements `IComparable<Book>`. `CompareTo` returns the result of `Price.CompareTo(other.Price)`, with any non-null book ordered after null. Price comparison is ascending; a negative result means cheaper, zero means the same price, and a positive result means more expensive. No ISBN tie-breaker is added. [4]

The application creates `catalog.OrderBy(b => b).ToList()`. The default comparer invokes `IComparable<Book>`, so this actually exercises the implemented interface. `OrderBy` is stable for equal keys, and `ToList` materializes a separate list. Anyone holding the original `List<Book>` sees its unchanged order 111, 222, 333. Callers using the sorted list see 222, 333, 111. Both lists initially reference the same immutable records, which is safe. Sorting catalog itself would change the order visible through every alias of that list. [5, 11]

Equality and ordering deliberately use different attributes because the assignment requires them: two equal ISBNs can compare as different prices, while different ISBNs with the same price compare as zero. Price-based `CompareTo` is therefore unsuitable for ISBN uniqueness in `SortedSet<Book>`; use `HashSet<Book>` with the ISBN equality rule instead. The price comparer remains suitable for catalogue display. [12]

Checked transitions and the stock invariant

The invariant is $0 \leq \text{StockCount} \leq \text{int.MaxValue}$. Construction rejects negative stock. Restock and Sell reject quantities less than or equal to zero. Sell also rejects a quantity greater than the current stock. Restock evaluates `checked(StockCount + quantity)`, which throws `OverflowException` if the sum exceeds `Int32` capacity instead of assigning a wrapped negative value. Each successful method returns this with `{ StockCount = ... }`; the original instance remains unchanged. [9]

Operation	Validation	Failure type
Construction	Stock cannot be negative; title and ISBN must be nonblank; price cannot be negative	<code>ArgumentException</code> or <code>ArgumentOutOfRangeException</code>
Restock	Quantity must be positive; checked addition must fit in <code>Int32</code>	<code>ArgumentOutOfRangeException</code> or <code>OverflowException</code>
Sell	Quantity must be positive	<code>ArgumentOutOfRangeException</code>
Sell	Quantity cannot exceed stock	<code>InvalidOperationException</code>

Why callers must capture the result

Returning a new record preserves the previous state for other consumers and makes a change explicit at the call site. A caller must use `book = book.Sell(quantity)`, store the result elsewhere, or replace the appropriate list element. Calling `book.Sell(1)` and discarding its result leaves `book` unchanged, although invalid requests still throw. This design does not automatically update a list or a database.

A public `init` accessor would prohibit ordinary later assignments but allow external code such as `book with { StockCount = -1 }`. That expression would bypass `Restock` and `Sell`, and the ordinary validating constructor would not revalidate the altered stock. `private init` closes this route. Compile-failure checks exercise direct assignment, an external `with` initializer, and an object initializer. [2, 10]

Complete implementation: Appendix A, `Book.cs`. The demo and automated checks show new references and unchanged source records.

Task 3 Catalogue report and duplicate handling

Sorted display and the duplicate experiment

Display order	Title	Price	Stock
222	Clean Code	35.50	2
333	The Pragmatic Programmer	40.00	6
111	Refactoring	45.00	4

The sorted list is copied before adding a separate Book with ISBN 111 and different price and stock. The existing ISBN 111 record and the added record are different objects but compare equal. Equality-based duplicate detection finds the repeated ISBN. Only the experiment list grows from three to four entries.

```
var naive = withDuplicate.Where(b => !b.Equals(duplicate)).ToList();
```

Equals compares ISBN, so the predicate is false for both ISBN 111 records. The naive result contains only Clean Code (222) and The Pragmatic Programmer (333). Its count is 2, not 3. This filter removes an entire equality class; it does not mean “remove the object I just appended”. [6]

```
var corrected = withDuplicate
    .Where(b => !ReferenceEquals(b, duplicate)).ToList();
```

ReferenceEquals tests object identity. Only the exact appended duplicate object fails this predicate, leaving three records. The original Refactoring object survives with price 45.00 and stock 4. The count falls by exactly one, and the original catalogue remains 111, 222, 333 with all supplied values. List<T>.Remove is not used. If the exact same object reference occurred twice, this predicate would exclude both occurrences; the demonstrated duplicate is a separately constructed object. [7]

State or filter	Count	Surviving ISBNs
Sorted catalogue	3	222, 333, 111
Experiment copy with duplicate	4	222, 333, 111, 111
Where with negated Equals	2	222, 333
Where with negated ReferenceEquals	3	222, 333, 111

Stock change demonstration

Starting from Refactoring with stock 4, Restock(3) returns stock 7; Sell(2) on that result returns stock 5. Both earlier records retain their values. Sell(6) is then rejected with “Cannot sell 6 copies; only 5 are in stock.” The last valid record keeps stock 5, while the catalogue record remains at 4. The complete, actual console transcript appears in the results appendix.

Complete implementation: Appendix A, Program.cs. Boundary cases, collection preservation and duplicate-filter results are checked in Checks.cs.

Verification and execution

Build and run steps

Install a .NET 10 SDK, then open PowerShell in the extracted source directory, which contains IFP3.csproj. The tested SDK was 10.0.401 on Windows. No NuGet packages or external services are required. Run these commands in order:

```
dotnet --version
dotnet build .\IFP3.csproj -c Release
dotnet run --project .\baseline\Baseline.csproj -c Release
dotnet run --project .\IFP3.csproj -c Release --no-build
dotnet run --project .\IFP3.csproj -c Release --no-build -- --check
powershell -NoProfile -ExecutionPolicy Bypass -File .\verify.ps1
```

The final command runs the verification workflow, including deliberate compilation failures for forbidden setters. These probe failures are expected evidence that the public API prevents mutation; the script itself must finish successfully. A successful build and a successful run both return exit code 0. The baseline prints 0. The application prints the task demonstrations, and the check runner reports passed checks.

Expected and actual results

Input or scenario	Expected	Actual result
Shared mutable alias	featured stock becomes 0	0
Immutable alias with captured Sell(4)	Source 4 and new value 0	4 and 0
Original and sorted catalogue ISBN order	111,222,333 / 222,333,111	Matches
Duplicate exclusion from four entries	Equals gives 2; identity gives 3	2 and 3
Initial stock -1	Reject construction	ArgumentOutOfRangeException
Zero or negative restock and sale	Reject quantity	ArgumentOutOfRangeException
Sell all available stock	Return stock 0; preserve source	Passed
Oversell by one	Reject; keep prior stock	InvalidOperationException
Restock beyond int.MaxValue	Reject the overflowing sum	OverflowException
Discard a valid Sell result	Original stock unchanged	Passed
External stock assignment and initializers	Compilation rejected	Rejected

The Release build completed with 0 warnings and 0 errors. All 43 runtime checks and all 6 compile-time guards passed. Actual evidence is captured in evidence/build.txt, evidence/baseline.txt, evidence/demo.txt, evidence/checks.txt and evidence/compile-guards.txt. Appendix B reproduces the console demonstration and check output. The verification also checks equality operators, hashing, null handling, price ties, valid integer boundaries and unchanged catalogue data.

Architecture and discussion

Structure and complexity

Book owns the data rules. Program composes the catalogue operations and prints evidence. DefaultBook provides an isolated comparison with generated equality. Checks supplies dependency-free automated assertions, while verify.ps1 exercises build, execution and compile-time access restrictions. The mutable baseline is a separate project so the two Book models cannot conflict.

Let n be the catalogue size and L the ISBN length. Stock operations and price comparisons take $O(1)$ time and allocate at most one fixed-size record, $O(1)$ extra space. ISBN equality is $O(L)$ in the worst case; hashing the ISBN is $O(L)$. Sorting and materializing the view takes $O(n \log n)$ time and $O(n)$ auxiliary space. Counting matches or filtering with Equals takes $O(nL)$ time; reference filtering takes $O(n)$. Counting uses $O(1)$ auxiliary space, whereas ToList needs $O(n)$ result storage. For fixed-size ISBNs, L can be treated as a constant. Printing takes time proportional to the amount of text produced.

Questions for oral defence

Why does the starter print 0? display and featured reference the same mutable object. The public setter at `display.StockCount = 0` changes that one object.

Does record mean immutable? No. A record class is still a reference type and can contain setters or mutable referenced children. Restricted accessors and immutable fields provide immutability here.

Why customize both equality and hashing? The identity rule is ISBN only. Equal ISBNs must produce equal hashes so hash-based collections use the same identity rule.

Why can CompareTo return zero for unequal books? Ordering uses price while equality uses ISBN. This is appropriate for the required display, but price-based ordered sets do not enforce ISBN identity.

What happens when a Sell result is ignored? A successful new record is discarded; the old variable and any list entry retain their original stock. A failing sale still throws.

Why is public init unsafe for stock? An external with initializer could set negative stock without the validation in Sell or Restock. The normal constructor is not re-run for that change.

Why does the first duplicate filter remove two books? Both objects have the same ISBN, so both satisfy `Equals(duplicate)`. `ReferenceEquals` selects only the separate appended object.

What small change could an instructor request? To display highest price first, change `OrderBy(b => b)` to `OrderByDescending(b => b)`. Keep `CompareTo` ascending and keep `ToList` so the original list order remains unchanged.

Conclusion

The redesign preserves the required dataset, prevents hidden changes through aliases, and makes stock changes explicit and validated. The experiments distinguish record equality, ISBN identity, price ordering and object identity. These distinctions explain both the original mutation bug and the duplicate-filtering trap.

Appendix A Complete source code

These listings reproduce the submitted files as readable text. Book.cs implements Tasks 1 and 2; Program.cs demonstrates all tasks. The baseline, comparison record, checks, verification script and project files make the submission reproducible.

Book model for Tasks 1 and 2

Book.cs

```
namespace Bookstore;

public sealed record Book : IComparable<Book>
{
    public string Title { get; }
    public string Isbn { get; }
    public decimal Price { get; }
    public int StockCount { get; private init; }

    public Book(string title, string isbn, decimal price, int stockCount)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(title);
        ArgumentException.ThrowIfNullOrWhiteSpace(isbn);
        ArgumentOutOfRangeException.ThrowIfNegative(price);
        ArgumentOutOfRangeException.ThrowIfNegative(stockCount);
        Title = title;
        Isbn = isbn;
        Price = price;
        StockCount = stockCount;
    }

    public bool Equals(Book? other) =>
        other is not null && StringComparer.Ordinal.Equals(Isbn, other.Isbn);

    public override int GetHashCode() => StringComparer.Ordinal.GetHashCode(Isbn);

    public int CompareTo(Book? other) => other is null ? 1 : Price.CompareTo(other.Price);

    public Book Restock(int quantity)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(quantity);
        return this with { StockCount = checked(StockCount + quantity) };
    }

    public Book Sell(int quantity)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(quantity);
        if (quantity > StockCount)
            throw new InvalidOperationException(
                $"Cannot sell {quantity} copies; only {StockCount} are in stock.");
        return this with { StockCount = StockCount - quantity };
    }
}
```

Default equality comparison for Task 1

DefaultBook.cs

```
namespace Bookstore;

internal sealed record DefaultBook(string Title, string Isbn, decimal Price, int StockCount);
```

Application for Tasks 1 to 3

Program.cs

```
using System.Globalization;

namespace Bookstore;

internal static class Program
{
    public static int Main(string[] args)
    {
        CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;
        if (args.Contains("--check"))
            return Checks.Run();

        List<Book> catalog = CreateCatalog();
        Console.WriteLine("TASK 1: immutable records and ISBN equality");
        Book featured = catalog[0];
        Book display = featured;
        Console.WriteLine($"Initially shared instance: {ReferenceEquals(featured, display)}");
        display = display.Sell(4);
        Console.WriteLine($"After display.Sell(4): featured={featured.StockCount}, " +
            $"display={display.StockCount}");
        Console.WriteLine($"Still shared instance: {ReferenceEquals(featured, display)}");
        featured.Sell(1);
        Console.WriteLine($"Discarded Sell(1): featured stock={featured.StockCount}");

        Console.WriteLine("Prediction: default record equality compares " +
            "all four stored properties.");
        var defaultA = new DefaultBook("Refactoring", "111", 45.00m, 4);
        var defaultB = new DefaultBook("Refactoring", "111", 99.00m, 9);
        var defaultCopy = new DefaultBook("Refactoring", "111", 45.00m, 4);
        Console.WriteLine("Default equality, same ISBN but changed price/stock: " +
            (defaultA == defaultB));
        Console.WriteLine($"Default equality, identical values: {defaultA == defaultCopy}");

        var duplicate = new Book("Refactoring", "111", 99.00m, 9);
        Console.WriteLine($"Custom equality, same ISBN: {featured.Equals(duplicate)}");
        Console.WriteLine($"Custom equality, different ISBN: {featured.Equals(catalog[1])}");
        Console.WriteLine("Equal ISBN gives equal hash: " +
            (featured.GetHashCode() == duplicate.GetHashCode()));

        Console.WriteLine("\nTASK 2: ascending price view using IComparable<Book>");
        List<Book> sorted = catalog.OrderBy(book => book).ToList();
        PrintCatalog(sorted);
        Console.WriteLine("Original order: " +
            string.Join(", ", catalog.Select(book => book.Isbn)));

        Console.WriteLine("\nTASK 3: duplicate exclusion and controlled stock changes");
        var withDuplicate = new List<Book>(sorted) { duplicate };
        int matches = withDuplicate.Count(book => book.Equals(duplicate));
        Console.WriteLine($"After adding ISBN 111: count={withDuplicate.Count}; " +
            $"equal matches={matches}");

        List<Book> naive = withDuplicate.Where(book => !book.Equals(duplicate)).ToList();
        Console.WriteLine($"Where(!Equals): count={naive.Count}; ISBNs=" +
            string.Join(", ", naive.Select(book => book.Isbn)));
        List<Book> corrected = withDuplicate
            .Where(book => !ReferenceEquals(book, duplicate)).ToList();
        Console.WriteLine($"Where(!ReferenceEquals): count={corrected.Count}; " +
            $"removed={withDuplicate.Count - corrected.Count}");
        Book preserved = corrected.Single(book => book.Isbn == "111");
        Console.WriteLine($"Original ISBN 111 preserved: price={preserved.Price:F2}, " +
            $"stock={preserved.StockCount}, " +
            $"same instance={ReferenceEquals(featured, preserved)}");

        Book before = catalog[0];
        Book restocked = before.Restock(3);
        Console.WriteLine($"Restock(3): before={before.StockCount}, " +
            $"after={restocked.StockCount}, " +
```

```

        $"new instance={!ReferenceEquals(before, restocked)}}");
    Book sold = restocked.Sell(2);
    Console.WriteLine($"Sell(2): before={restocked.StockCount}, after={sold.StockCount}, " +
        $"new instance={!ReferenceEquals(restocked, sold)}}");
    try
    {
        sold.Sell(6);
    }
    catch (InvalidOperationException exception)
    {
        Console.WriteLine($"Oversell rejected: {exception.Message}");
    }
    Console.WriteLine($"After rejection: stock={sold.StockCount}");
    Console.WriteLine("Original catalogue unchanged: " +
        string.Join(", ", catalog.Select(book => $"{book.Isbn}:{book.StockCount}")));
    return 0;
}

internal static List<Book> CreateCatalog() => new()
{
    new Book("Refactoring", "111", 45.00m, 4),
    new Book("Clean Code", "222", 35.50m, 2),
    new Book("The Pragmatic Programmer", "333", 40.00m, 6),
};

private static void PrintCatalog(IEnumerable<Book> books)
{
    foreach (Book book in books)
        Console.WriteLine($"{book.Isbn} | {book.Title} | {book.Price:F2} | " +
            $"stock={book.StockCount}");
}
}

```

Automated checks

Checks.cs

```

using System.Diagnostics.CodeAnalysis;

namespace Bookstore;

internal static class Checks
{
    private static int passed;

    private static void Check([DoesNotReturnIf(false)] bool condition, string description)
    {
        if (!condition) throw new InvalidOperationException(description);
        Console.WriteLine($"PASS {++passed:00}: {description}");
    }

    private static void Reject<T>(Action action, string description) where T : Exception
    {
        try { action(); }
        catch (T) { Check(true, description); return; }
        throw new InvalidOperationException($"{description}: expected {typeof(T).Name}.");
    }

    public static int Run()
    {
        passed = 0;
        try
        {
            var original = new Book("Refactoring", "111", 45.00m, 4);
            var duplicate = new Book("Different title", "111", 1.00m, 99);
            var other = new Book("Refactoring", "999", 45.00m, 4);
            Check(original.Equals(duplicate),
                "Equality ignores title, price and stock for the same ISBN");
            Check(!original.Equals(other), "Different ISBNs are unequal");
            Check(original == duplicate && original != other, "Record operators use custom equality");
            Check(((object)original).Equals(duplicate), "Object.Equals dispatches to ISBN equality");
        }
        catch { }
    }
}

```

```

Check(original.GetHashCode() == duplicate.GetHashCode(),
    "Equal ISBNs produce equal hashes");
Check(new HashSet<Book> { original, duplicate, other }.Count == 2,
    "HashSet deduplicates by ISBN");
Check(!EqualityComparer<Book>.Default.Equals(original, null) && !original.Equals("111"),
    "Null and another type are unequal");
var plain = new DefaultBook("Refactoring", "111", 45.00m, 4);
Check(plain == new DefaultBook("Refactoring", "111", 45.00m, 4),
    "Default record equality accepts identical components");
Check(plain != (plain with { Title = "Changed" }) &&
    plain != (plain with { Isbn = "999" }) &&
    plain != (plain with { Price = 1m }) &&
    plain != (plain with { StockCount = 99 })),
    "Default record equality includes all four components");
Check(original.CompareTo(duplicate) > 0 && duplicate.CompareTo(original) < 0,
    "Ordering uses ascending price even for equal ISBNs");
Check(original.CompareTo(other) == 0,
    "Equal prices compare as a tie despite unequal ISBNs");
Check(original.CompareTo(null) > 0, "A book compares greater than null");
Check(new Book("Zero", "0", 0m, 0).StockCount == 0,
    "Zero price and zero initial stock are valid");
Reject<ArgumentOutOfRangeException>(() => new Book("X", "x", 1m, -1),
    "Negative initial stock is rejected");
Reject<ArgumentOutOfRangeException>(() => new Book("X", "x", 1m, int.MinValue),
    "Minimum integer initial stock is rejected");
Reject<ArgumentException>(() => new Book(" ", "x", 1m, 0), "Blank title is rejected");
Reject<ArgumentException>(() => new Book("X", " ", 1m, 0), "Blank ISBN is rejected");
Reject<ArgumentOutOfRangeException>(() => new Book("X", "x", -1m, 0),
    "Negative initial price is rejected");
foreach (int quantity in new[] { 0, -1, int.MinValue })
{
    Reject<ArgumentOutOfRangeException>(() => original.Restock(quantity),
        $"Restock({quantity}) rejects a nonpositive quantity");
    Reject<ArgumentOutOfRangeException>(() => original.Sell(quantity),
        $"Sell({quantity}) rejects a nonpositive quantity");
}
var restocked = original.Restock(3);
var sold = restocked.Sell(2);
Check(restocked.StockCount == 7 && sold.StockCount == 5,
    "Normal restock and sale calculate stock");
Check(!ReferenceEquals(original, restocked) && !ReferenceEquals(restocked, sold),
    "Each stock operation returns a new reference");
Check(original.StockCount == 4 && restocked.StockCount == 7,
    "Earlier references retain their original stock");
original.Sell(1);
original.Restock(1);
Check(original.StockCount == 4,
    "Discarding either returned record leaves the source unchanged");
var empty = original.Sell(4);
Check(empty.StockCount == 0, "Selling all available stock reaches zero");
Reject<InvalidOperationException>(() => original.Sell(5), "Overselling is rejected");
Reject<InvalidOperationException>(() => empty.Sell(1),
    "Selling from empty stock is rejected");
var maximum = original.Restock(int.MaxValue - 4);
Check(maximum.StockCount == int.MaxValue, "Restock can reach the maximum integer exactly");
Reject<OverflowException>(() => maximum.Restock(1),
    "Restock beyond maximum stock is rejected");
Reject<OverflowException>(() => original.Restock(int.MaxValue),
    "Large positive restock cannot wrap");
Check(original.StockCount == 4 && maximum.StockCount == int.MaxValue,
    "Rejected operations leave source records unchanged");
Check(maximum.Sell(int.MaxValue).StockCount == 0,
    "Maximum positive sale is valid when fully stocked");
var catalog = Program.CreateCatalog();
var before = catalog.Select(b => (b.Title, b.Isbn, b.Price, b.StockCount)).ToArray();
var expected = new[]
{
    ("Refactoring", "111", 45.00m, 4),
    ("Clean Code", "222", 35.50m, 2),
    ("The Pragmatic Programmer", "333", 40.00m, 6)
}

```

```

    };
    Check(before.SequenceEqual(expected),
        "Required catalogue has exact original order and data");
    var sorted = catalog.OrderBy(b => b).ToList();
    Check(sorted.Select(b => b.Isbn).SequenceEqual(new[] { "222", "333", "111" })),
        "Sorted view orders the required catalogue cheapest first");
    Check(catalog.Select(b => (b.Title, b.Isbn, b.Price, b.StockCount)).SequenceEqual(before),
        "Building a sorted view preserves original order and every field");
    var copy = sorted.ToList();
    copy.Add(duplicate);
    Check(copy.Count == 4 && copy.Count(b => b.Equals(duplicate)) == 2,
        "Copied catalogue contains two ISBN-equal entries");
    var naive = copy.Where(b => !b.Equals(duplicate)).ToList();
    Check(naive.Count == 2 && naive.All(b => b.Isbn != "111"),
        "Equality filtering removes both the original and duplicate (4 to 2)");
    var corrected = copy.Where(b => !ReferenceEquals(b, duplicate)).ToList();
    Check(corrected.Count == 3 && corrected.Any(b => ReferenceEquals(b, catalog[0])) &&
        !corrected.Any(b => ReferenceEquals(b, duplicate)),
        "Reference filtering removes only the inserted object (4 to 3)");
    Check(corrected.Single(b => b.Isbn == "111").Price == 45.00m &&
        corrected.Single(b => b.Isbn == "111").StockCount == 4 &&
        catalog.Select(b => (b.Title, b.Isbn, b.Price, b.StockCount)).SequenceEqual(before),
        "Duplicate filtering preserves the original price, stock and catalogue");
    Console.WriteLine($"All {passed} checks passed.");
    return 0;
}
catch (Exception error)
{
    Console.Error.WriteLine($"FAIL after {passed} passes: {error}");
    return 1;
}
}
}

```

Verification script

verify.ps1

```

$ErrorActionPreference = 'Stop'
$env:DOTNET_CLI_UI_LANGUAGE = 'en'
$project = Join-Path $PSScriptRoot 'IFP3.csproj'
$evidence = Join-Path $PSScriptRoot 'evidence'
New-Item -ItemType Directory -Force -Path $evidence | Out-Null
& dotnet build $project -c Release --nologo
if ($LASTEXITCODE -ne 0) { throw 'Application build failed.' }
$checks = & dotnet run --project $project -c Release --no-build -- --check 2>&1
$checkExit = $LASTEXITCODE
$checks | Set-Content (Join-Path $evidence 'checks.txt') -Encoding UTF8
$checks | Write-Output
if ($checkExit -ne 0) { throw 'Runtime checks failed.' }

$tempRoot = [IO.Path]::GetFullPath([IO.Path]::GetTempPath()).TrimEnd('\')
$probe = Join-Path $tempRoot ('ifp3-guards-' + [Guid]::NewGuid().ToString('N'))
New-Item -ItemType Directory -Path $probe | Out-Null
try {
    [xml]$projectXml = Get-Content -LiteralPath $project -Raw
    $target = $projectXml.Project.PropertyGroup.TargetFramework
    $reference = [Security.SecurityElement]::Escape($project)
    $probeProject = Join-Path $probe 'Probe.csproj'
    @"
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType><TargetFramework>$target</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings><Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup><ProjectReference Include="$reference" /></ItemGroup>
</Project>
"@ | Set-Content -LiteralPath $probeProject -Encoding UTF8
    $source = Join-Path $probe 'Probe.cs'
    $prefix = "using Bookstore;`r`nvar original = new Book(`"X`", `"x`", 1m, 0);`r`n"
    ($prefix + 'Console.WriteLine(original.StockCount);') | Set-Content $source -Encoding UTF8
}

```

```

    $positive = & dotnet build $probeProject -c Release --nologo 2>&1
    if ($LASTEXITCODE -ne 0) { throw "Positive compile control failed: $positive" }
    $invalid = @'
original.StockCount = -1;
_ = original with { StockCount = -1 };
_ = new Book("X", "x", 1m, 0) { StockCount = -1 };
_ = original with { Title = "Changed" };
_ = original with { Isbn = "changed" };
_ = original with { Price = -1m };
'@
    ($prefix + $invalid) | Set-Content $source -Encoding UTF8
    $diagnostics = & dotnet build $probeProject -c Release --nologo 2>&1
    $negativeExit = $LASTEXITCODE
    $log = @('Positive compile control: PASS', "Invalid caller compiler exit: $negativeExit")
    $log += $diagnostics
    if ($negativeExit -eq 0) { throw 'Invalid public writes unexpectedly compiled.' }
    foreach ($line in 3..8) {
        if (($diagnostics -join "`n") -notmatch "Probe.cs\$line,\d+\): error CS(0272|8852|0200)") {
            throw "Expected an access/readonly error on probe line $line. $diagnostics"
        }
    }
    $log += 'All 6 compile guards passed (expected errors).'
    $log | Set-Content (Join-Path $evidence 'compile-guards.txt') -Encoding UTF8
    Write-Output $log[-1]
}
finally {
    $resolvedProbe = [IO.Path]::GetFullPath($probe)
    if ([IO.Path]::GetDirectoryName($resolvedProbe) -ne $tempRoot -or
        [IO.Path]::GetFileName($resolvedProbe) -notlike 'ifp3-guards-*') {
        throw "Refusing cleanup outside the verification temp directory: $resolvedProbe"
    }
    Remove-Item -LiteralPath $resolvedProbe -Recurse -Force
}

```

Main project configuration

IFP3.csproj

```

<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <Compile Remove="baseline/**/*.cs;work/**/*.cs;output/**/*.cs" />
  </ItemGroup>

</Project>

```

Supplied starter

baseline/OriginalSnippet.txt

```
class Book
{
    public string Title { get; set; }
    public string Isbn { get; set; }
    public decimal Price { get; set; }
    public int StockCount { get; set; }
}

List<Book> catalog = new()
{
    new Book { Title = "Refactoring", Isbn = "111", Price = 45.00m, StockCount = 4 },
    new Book { Title = "Clean Code", Isbn = "222", Price = 35.50m, StockCount = 2 },
    new Book { Title = "The Pragmatic Programmer", Isbn = "333", Price = 40.00m, StockCount = 6 },
};

Book featured = catalog[0];
Book display = featured;
display.StockCount = 0;

Console.WriteLine(featured.StockCount);
```

Runnable baseline wrapper

baseline/Program.cs

```
class Book
{
    public string Title { get; set; }
    public string Isbn { get; set; }
    public decimal Price { get; set; }
    public int StockCount { get; set; }
}

internal static class Program
{
    public static void Main()
    {
        List<Book> catalog = new()
        {
            new Book { Title = "Refactoring", Isbn = "111", Price = 45.00m, StockCount = 4 },
            new Book { Title = "Clean Code", Isbn = "222", Price = 35.50m, StockCount = 2 },
            new Book { Title = "The Pragmatic Programmer", Isbn = "333", Price = 40.00m, StockCount = 6 },
        },

        };

        Book featured = catalog[0];
        Book display = featured;
        display.StockCount = 0;

        Console.WriteLine(featured.StockCount);
    }
}
```

Baseline project configuration

baseline/Baseline.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
  </PropertyGroup>
</Project>
```

Appendix B Actual execution evidence

Mutable baseline output

0

Application output

TASK 1: immutable records and ISBN equality
Initially shared instance: True
After display.Sell(4): featured=4, display=0
Still shared instance: False
Discarded Sell(1): featured stock=4
Prediction: default record equality compares all four stored properties.
Default equality, same ISBN but changed price/stock: False
Default equality, identical values: True
Custom equality, same ISBN: True
Custom equality, different ISBN: False
Equal ISBN gives equal hash: True

TASK 2: ascending price view using IComparable<Book>
222 | Clean Code | 35.50 | stock=2
333 | The Pragmatic Programmer | 40.00 | stock=6
111 | Refactoring | 45.00 | stock=4
Original order: 111, 222, 333

TASK 3: duplicate exclusion and controlled stock changes
After adding ISBN 111: count=4; equal matches=2
Where(!Equals): count=2; ISBNs=222, 333
Where(!ReferenceEquals): count=3; removed=1
Original ISBN 111 preserved: price=45.00, stock=4, same instance=True
Restock(3): before=4, after=7, new instance=True
Sell(2): before=7, after=5, new instance=True
Oversell rejected: Cannot sell 6 copies; only 5 are in stock.
After rejection: stock=5
Original catalogue unchanged: 111:4, 222:2, 333:6

Automated check output

PASS 01: Equality ignores title, price and stock for the same ISBN
PASS 02: Different ISBNs are unequal
PASS 03: Record operators use custom equality
PASS 04: Object.Equals dispatches to ISBN equality
PASS 05: Equal ISBNs produce equal hashes
PASS 06: HashSet deduplicates by ISBN
PASS 07: Null and another type are unequal
PASS 08: Default record equality accepts identical components
PASS 09: Default record equality includes all four components
PASS 10: Ordering uses ascending price even for equal ISBNs
PASS 11: Equal prices compare as a tie despite unequal ISBNs
PASS 12: A book compares greater than null
PASS 13: Zero price and zero initial stock are valid
PASS 14: Negative initial stock is rejected
PASS 15: Minimum integer initial stock is rejected
PASS 16: Blank title is rejected
PASS 17: Blank ISBN is rejected
PASS 18: Negative initial price is rejected
PASS 19: Restock(0) rejects a nonpositive quantity
PASS 20: Sell(0) rejects a nonpositive quantity
PASS 21: Restock(-1) rejects a nonpositive quantity
PASS 22: Sell(-1) rejects a nonpositive quantity
PASS 23: Restock(-2147483648) rejects a nonpositive quantity
PASS 24: Sell(-2147483648) rejects a nonpositive quantity
PASS 25: Normal restock and sale calculate stock
PASS 26: Each stock operation returns a new reference
PASS 27: Earlier references retain their original stock

PASS 28: Discarding either returned record leaves the source unchanged
 PASS 29: Selling all available stock reaches zero
 PASS 30: Overselling is rejected
 PASS 31: Selling from empty stock is rejected
 PASS 32: Restock can reach the maximum integer exactly
 PASS 33: Restock beyond maximum stock is rejected
 PASS 34: Large positive restock cannot wrap
 PASS 35: Rejected operations leave source records unchanged
 PASS 36: Maximum positive sale is valid when fully stocked
 PASS 37: Required catalogue has exact original order and data
 PASS 38: Sorted view orders the required catalogue cheapest first
 PASS 39: Building a sorted view preserves original order and every field
 PASS 40: Copied catalogue contains two ISBN-equal entries
 PASS 41: Equality filtering removes both the original and duplicate (4 to 2)
 PASS 42: Reference filtering removes only the inserted object (4 to 3)
 PASS 43: Duplicate filtering preserves the original price, stock and catalogue
 All 43 checks passed.

Compile time guard results

The valid control caller compiled successfully. The invalid caller failed with exit code 1 and six errors. The following diagnostic codes are extracted from the actual compiler log; full messages and paths are retained in evidence/compile-guards.txt.

Probe line	Forbidden operation	Actual diagnostic
3	Assign StockCount after construction	CS0200
4	Set StockCount in an external with expression	CS0200
5	Set StockCount in an object initializer	CS0200
6	Change Title in an external with expression	CS0200
7	Change Isbn in an external with expression	CS0200
8	Change Price in an external with expression	CS0200

All 6 compile guards passed (expected errors).

References

[1] Microsoft Learn. Records in C#.

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/record>

[2] Microsoft Learn. The with expression.

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/with-expression>

[3] Microsoft Learn. Object.GetHashCode method.

<https://learn.microsoft.com/en-us/dotnet/api/system.object.gethashcode>

[4] Microsoft Learn. IComparable<T>.CompareTo method.

<https://learn.microsoft.com/en-us/dotnet/api/system.icomparable-1.compareto>

[5] Microsoft Learn. Enumerable.OrderBy method.

<https://learn.microsoft.com/en-us/dotnet/api/system.linq.enumerable.orderby>

[6] Microsoft Learn. Enumerable.Where method.

<https://learn.microsoft.com/en-us/dotnet/api/system.linq.enumerable.where>

[7] Microsoft Learn. Object.ReferenceEquals method.

<https://learn.microsoft.com/en-us/dotnet/api/system.object.referenceequals>

[8] Microsoft Learn. Resolve errors related to the Main method and top level statements.

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/compiler-messages/entry-point-errors>

[9] Microsoft Learn. Checked and unchecked statements.

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/checked-and-unchecked>

[10] Microsoft Learn. The init keyword.

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/init>

[11] Microsoft Learn. Comparer<T>.Default property.

<https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.comparer-1.default>

[12] Microsoft Learn. IEquatable<T> interface.

<https://learn.microsoft.com/en-us/dotnet/api/system.ieuatable-1>

Assignment source: Assignment 3 Week 3, Implementing Interfaces and Programming with a Mutable Type/Immutable Models, supplied task sheet.